

DISEÑO DE INTERFACES DE PROGRAMACIÓN AVANZADO

LABORATORIO N° 7

TÉCNICAS AVANZADAS DE PROMPTING



PROMPTS FEW-SHOT · CHAIN OF THOUGHT · ROL · DESCOMPOSICIÓN

Alumnos	Joel Benjamin Romero Ramirez	Nota
Grupo	C	
Fecha de Entrega	01/10/26	
Docente	Jeisson Daniel Paredes Cano	

OBJETIVOS:

- Explicar con tus propias palabras qué son zero-shot, one-shot, few-shot, chain of thought, role prompting, descomposición, prompt estructurado y autocrítica.
- Comprobar en la práctica cómo los ejemplos (few-shot) fijan el formato de las respuestas de una IA.
- Comparar una respuesta directa con una respuesta razonada paso a paso (chain of thought) y verificar el resultado por tu cuenta.
- Aplicar roles, descomposición, prompts estructurados y autocrítica en tareas de desarrollo de software.
- Documentar tus experimentos en Markdown y publicarlos en tu repositorio de GitHub.

SEGURIDAD:

	Advertencia: En este laboratorio está prohibida la manipulación del hardware, conexiones eléctricas o de red; así como la ingestión de alimentos o bebidas. No compartas contraseñas, datos personales ni código confidencial en las herramientas de IA.
---	--

FUNDAMENTO TEÓRICO:

- Revisar el material de la Semana 7 que está en el Campus Virtual.
- Repasar la anatomía de un prompt de la Semana 6: rol, instrucción, contexto, ejemplos y formato.
- Si no hiciste el Laboratorio 06, revisa en él cómo se crea un repositorio y cómo se escribe en Markdown: aquí se usan los mismos pasos.

NORMAS EMPLEADAS:

- No aplica.

RECURSOS:

Todas las herramientas de este laboratorio son gratuitas. No necesitas tarjeta de crédito ni ningún plan de pago.

Herramienta	Para qué se usa	Costo / requisito
Asistente de IA: ChatGPT, Gemini, Claude o Copilot	Probar los prompts (Ejercicios 2 al 6 y la tarea)	Plan gratuito. Solo requiere crear una cuenta.
Calculadora (la de tu computadora o celular)	Comprobar el resultado del Ejercicio 3	Gratuita.
Git	Registrar y subir los cambios	Gratuito (git-scm.com).
Visual Studio Code	Escribir la bitácora con vista previa	Gratuito (code.visualstudio.com).
GitHub	Publicar el repositorio	Cuenta gratuita.
JDK 11 o superior (Eclipse Temurin)	Compilar el código del Ejercicio 5 (opcional)	Gratuito (adoptium.net).

METODOLOGÍA PARA EL DESARROLLO DE LA TAREA:

- El desarrollo del laboratorio es individual. La parte guiada se resuelve paso a paso junto con el docente; la tarea se desarrolla de forma autónoma.
- Todo se hace desde cero en este laboratorio: creas un repositorio nuevo llamado lab07-tecnicas y en él documentas, en una bitácora, todo lo que pruebes con la IA.

PROCEDIMIENTO:

Nota:

Las secciones en azul y cursiva brindan una explicación teórica o del código

PARTE GUIADA — Sigue las instrucciones paso a paso; el docente avanza contigo y resuelve dudas sobre la marcha.

Requisitos previos:

- Una cuenta gratuita en un asistente de IA (ChatGPT, Gemini, Claude o Copilot) con la sesión iniciada.
- Una cuenta gratuita en GitHub con la sesión iniciada en github.com.
- Git instalado: compruébalo escribiendo git --version en la terminal.
- Visual Studio Code instalado.

- Opcional: JDK 11 o superior para compilar el código del Ejercicio 5 (compruébalo con `java -version`).

Lo que vas a hacer:

En el Laboratorio 06 aprendiste a escribir un buen prompt. Hoy vas a probar técnicas para tareas más difíciles: dar ejemplos, pedir que la IA razone paso a paso, asignarle un rol, dividir una tarea grande, ordenar el prompt con etiquetas y pedirle que revise su propia respuesta. Con cada técnica harás lo mismo que la semana pasada: cambiar una sola cosa, observar qué cambia en la respuesta y anotarlo.

El mismo pedido se puede escribir de dos maneras. No las confundas:

Tipo de prompt	Qué tiene
Prompt básico	Una instrucción directa. Funciona bien en tareas sencillas, pero en tareas difíciles la IA puede equivocarse o responder de forma desordenada.
Prompt con técnicas avanzadas	Además de los cinco componentes, guía cómo debe trabajar la IA: con ejemplos, paso a paso, con un rol concreto, por partes o revisando su respuesta.

Siete palabras nuevas, una frase cada una:

Palabra	Qué significa
Shot	Ejemplo. Zero-shot = 0 ejemplos, one-shot = 1 ejemplo, few-shot = varios ejemplos (de 2 a 5).
Few-shot	Dar ejemplos de la respuesta que quieres para que la IA imite el mismo formato.
Chain of Thought	Cadena de pensamiento: pedir a la IA que razone paso a paso antes de dar la respuesta final.
Role prompting	Asignarle a la IA un rol concreto, por ejemplo "revisor de código Java senior".
Descomposición	Dividir una tarea grande en pasos pequeños y pedirlos en orden.
Prompt estructurado	Separar las partes del prompt con etiquetas (<code><rol>...</rol></code>) o títulos (<code>## Rol</code>).
Autocrítica	Pedir a la IA que revise su propia respuesta y corrija lo que falte.

Ejercicio 1: Crear tu repositorio y preparar la bitácora

Igual que en el Laboratorio 06, antes de hablar con la IA vas a crear el repositorio y el archivo donde registrarás todos tus experimentos. Si ya lo hiciste la semana pasada, estos pasos te tomarán pocos minutos.

1. Crear el repositorio en GitHub. Entra a github.com, haz clic en el botón New y completa el formulario:

Campo	Qué debe decir
Repository name	lab07-tecnicas
Description	Bitacora de tecnicas avanzadas de prompting
Public / Private	Public

Campo	Qué debe decir
Add a README file	Marcado.

Haz clic en el botón verde Create repository.

Resultado esperado: *El repositorio se abre y ya muestra un archivo README.md con el nombre lab07-tecnicas.*

2. Comprobar tu identidad en Git. En la terminal:

```
git config --global user.name
git config --global user.email
```

Si configuraste Git en el Laboratorio 06, verás tu nombre y tu correo y puedes pasar al paso 3. Si alguno no muestra nada, configúralo como en el laboratorio anterior: git config --global user.name "Tu Nombre Apellido" y git config --global user.email "tucorreo@ejemplo.com".

3. Clonar el repositorio en tu computadora. En la terminal, ubícate en la carpeta donde guardas tus trabajos:

```
cd E:\TECSUP
git clone https://github.com/TU-USUARIO/lab07-tecnicas.git
cd lab07-tecnicas
code .
```

Reemplaza TU-USUARIO por tu nombre de usuario de GitHub. Usa una carpeta que no esté dentro de otro repositorio Git (por ejemplo, no dentro de lab06-prompts).

Resultado esperado: *Cloning into 'lab07-tecnicas'... seguido de done, y Visual Studio Code se abre con el archivo README.md.*

4. Crear la carpeta de la bitácora. En la terminal:

```
mkdir prompts
```

5. Crear el archivo **prompts/BITACORA.md** desde el panel izquierdo de VS Code y escribir esta estructura:

```
# Bitacora de tecnicas avanzadas

Laboratorio 07: Tecnicas Avanzadas de Prompting.
Herramienta de IA usada: (escribe aqui cual usaste)

## Ejercicio 2: Zero-shot, one-shot y few-shot

## Ejercicio 3: Chain of Thought

## Ejercicio 4: Role prompting

## Ejercicio 5: Descomposicion

## Ejercicio 6: Prompt estructurado y autocritica
```

Recuerda los símbolos de Markdown del Laboratorio 06: # para el título, ## para cada sección, | para las filas de una tabla y ```text ... ``` para guardar un prompt tal cual.

6. Abrir la vista previa con Ctrl + Shift + V y dejarla al costado.
7. Abrir tu asistente de IA en el navegador, con la sesión iniciada. Deja el navegador y VS Code lado a lado.

Resultado esperado: La vista previa muestra un título grande y cinco secciones vacías, una por cada ejercicio.

Cuidado: Nunca pegues en una IA contraseñas, datos personales de otras personas ni código confidencial de una empresa. Todo lo que escribes puede quedar almacenado.

Cuidado: Usa siempre un chat nuevo cuando el ejercicio lo indique. Si pruebas dos prompts en el mismo chat, el primero queda en la ventana de contexto y cambia la respuesta del segundo.

Captura del resultado:



Ejercicio 2: Zero-shot, one-shot y few-shot

"Shot" significa ejemplo. Vas a pedirle a la IA la misma tarea tres veces: sin ejemplos, con un ejemplo y con varios ejemplos. La tarea es clasificar comentarios de clientes, es decir, decidir si cada uno es Positivo, Negativo o Neutral.

1. Zero-shot (sin ejemplos). En un chat nuevo envía:

Clasifica estos comentarios de clientes:

1. Me encanto, llego rapido
2. No lo recomiendo
3. Es aceptable por el precio
4. Pesima atencion, no vuelvo
5. Excelente calidad, lo volveria a comprar

2. One-shot (un ejemplo). En otro chat nuevo envía:

Clasifica cada comentario como Positivo, Negativo o Neutral.
Ejemplo:
"Me gusto mucho" -> Positivo

Comentarios:

1. Me encanto, llego rapido
2. No lo recomiendo
3. Es aceptable por el precio
4. Pesima atencion, no vuelvo
5. Excelente calidad, lo volveria a comprar

3. Few-shot (varios ejemplos). En otro chat nuevo envía:

Clasifica cada comentario. Responde solo con el formato de los ejemplos.
"Me gusto mucho" -> Positivo
"Que decepcion" -> Negativo
"Esta bien, nada especial" -> Neutral

"Me encanto, llego rapido" ->
"No lo recomiendo" ->
"Es aceptable por el precio" ->
"Pesima atencion, no vuelvo" ->
"Excelente calidad, lo volveria a comprar" ->

Fíjate en que los ejemplos tienen un comentario de cada tipo y todos usan el mismo formato: texto entre comillas, una flecha y la etiqueta. Eso es lo que la IA va a imitar.

Resultado esperado: Las etiquetas correctas son: Positivo, Negativo, Neutral, Negativo, Positivo. En zero-shot la IA suele acertar, pero con un formato libre (a veces con explicaciones). En few-shot responde línea por línea con el formato "texto -> etiqueta".

4. Registrar los resultados. Agrega esta tabla en la sección del Ejercicio 2 de tu bitácora y complétala:

Tipo	Aciertos (de 5)	Formato de la respuesta	Todas con el mismo formato (Si/No)
Zero-shot			
One-shot			
Few-shot			

Cuidado: Que la IA acierte en zero-shot no significa que few-shot no sirva. El objetivo de few-shot es que el formato sea siempre el mismo, por ejemplo para copiar las 500 respuestas a Excel sin corregirlas una por una.

Captura del resultado:

Ejercicio 2: Zero-shot, one-shot y few-shot

Tipo	Aciertos (de 5)	Formato de la respuesta	Todas con el mismo formato (Si/No)
Zero-shot	5	NINGUNO	NO
One-shot	5	ESTRUCTURADO	SI
Few-shot	5	ESTRUCTURADO	SI

Ejercicio 3: Chain of Thought (paso a paso)

Chain of Thought significa cadena de pensamiento: le pides a la IA que escriba su razonamiento por pasos antes de dar la respuesta. Así puedes comprobar cada paso en lugar de solo creerle.

1. Pedido directo. En un chat nuevo envía:

Un producto cuesta S/ 120. La tienda aplica un descuento del 25 % y luego suma el 18 % de IGV sobre el precio con descuento.
Un cliente compra 3 unidades. Cuanto paga en total?
Responde solo con el numero.

2. Pedido paso a paso. En otro chat nuevo envía:

Un producto cuesta S/ 120. La tienda aplica un descuento del 25 % y luego suma el 18 % de IGV sobre el precio con descuento.
Un cliente compra 3 unidades. Cuanto paga en total?
Resuélvelo paso a paso: muestra cada calculo y comprueba el resultado antes de dar la respuesta final.

3. Comprobar tú mismo con la calculadora, en este orden:

Paso	Cálculo	Resultado
1. Precio con descuento	$120 \times 0,75$	90
2. Precio con IGV	$90 \times 1,18$	106,20
3. Total por 3 unidades	$106,20 \times 3$	318,60

Resultado esperado: El total correcto es S/ 318,60. En el pedido paso a paso la IA muestra cada cálculo y puedes compararlo con la tabla. En el pedido directo solo ves un número.

Cuidado: Es posible que la IA acierte también en el pedido directo. No importa: la ventaja de Chain of Thought es que puedes revisar cada paso y, si hay un error, saber exactamente dónde está.

4. Registrar los resultados en la sección del Ejercicio 3 de tu bitácora:

Pedido	Respuesta de la IA	Muestra los pasos (Si/No)	Correcta (Si/No)
Directo			
Paso a paso			

Debajo de la tabla escribe en dos líneas por qué es útil ver el razonamiento aunque la respuesta directa haya sido correcta.

Captura del resultado:

Ejercicio 3: Chain of Thought

Pedido	Respuesta de la IA	Muestra los pasos (Si/No)	Correcta (Si/No)
Directo	318.60	NO	SI
Paso a paso	Muestra detalladamente todos los pasos hasta llegar a la solución	SI	SI

Un producto cuesta S/ 120. La tienda aplica un descuento del 25 %
y luego suma el 18 % de IGV sobre el precio con descuento.
Un cliente compra 3 unidades. Cuanto paga en total?
Responde solo con el numero.

318.60

Un producto cuesta S/ 120. La tienda aplica un descuento del 25 %
y luego suma el 18 % de IGV sobre el precio con descuento.
Un cliente compra 3 unidades. Cuanto paga en total? 

Aquí tienes la resolución paso a paso del problema:

Paso 1: Calcular el precio con el 25% de descuento por unidad

- Precio original: S/ 120
- Monto del descuento (25%): $120 \times 0.25 = \text{S/ } 30$
- Precio con descuento: $120 - 30 = \text{S/ } 90$
(Comprobación alternativa: $120 \times 0.75 = \text{S/ } 90$)

Paso 2: Calcular el precio con el 18% de IGV

- Precio base (con descuento): S/ 90
- Monto del IGV (18%): $90 \times 0.18 = \text{S/ } 16.20$
- Precio final por unidad: $90 + 16.20 = \text{S/ } 106.20$
(Comprobación alternativa: $90 \times 1.18 = \text{S/ } 106.20$)

Paso 3: Calcular el total por 3 unidades

- Cantidad: 3 unidades
- Precio unitario final: S/ 106.20
- Cálculo: $106.20 \times 3 = \text{S/ } 318.60$

Comprobación final

- Precio total de las 3 unidades antes de impuestos: $90 \times 3 = \text{S/ } 270$
- Aplicando el IGV del 18% al monto total: $270 \times 1.18 = \text{S/ } 318.60$

Ambos métodos coinciden perfectamente.

Respuesta final: El cliente paga en total S/ 318.60.

Ejercicio 4: Role prompting (asignar un rol)

El rol le indica a la IA desde qué punto de vista responder. No le da conocimientos nuevos: cambia el enfoque, el vocabulario y el nivel de detalle. Vas a hacer la misma pregunta con tres roles distintos.

1. Enviar los tres prompts, cada uno en un chat nuevo:

Versión	Prompt
A. Sin rol	Explica que es una variable en programación.
B. Rol docente	Actúa como profesor de programación que explica a estudiantes que nunca han programado. Explica que es una variable en programación.
C. Rol senior	Actúa como desarrollador Java senior que explica a un compañero de trabajo. Explica que es una variable en programación.

Resultado esperado: La versión B usa palabras sencillas y comparaciones de la vida diaria (por ejemplo, una caja con etiqueta). La versión C usa términos técnicos como tipo de dato, memoria o alcance, y probablemente código Java.

2. Registrar las diferencias en la sección del Ejercicio 4 de tu bitácora:

```
| Version | Vocabulario (sencillo/técnico) | Usa ejemplos o código | A quien le
sirve más |
|-----|-----|-----|-----|
| A. Sin rol | | | |
| B. Rol docente | | | |
| C. Rol senior | | | |
```

Cuidado: Un rol vago como "Actúa como experto" casi no cambia la respuesta. Di siempre qué experto y para quién explica.

Captura del resultado:

Ejercicio 4: Role prompting			
Versión	Vocabulario (sencillo/técnico)	Usa ejemplos o código	A quien le sirve más
A. Sin rol	SENCILLO	NO	ESTUDIANTE
B. Rol docente	SENCILLO	NO	ESTUDIANTE
C. Rol senior	TÉCNICO	SI	JUNIOR

Ejercicio 5: Descomposición (dividir la tarea)

Cuando pides algo grande de una sola vez, la IA responde de forma general. Si divides la tarea en pasos y los pides en orden, puedes revisar cada parte antes de seguir. Aquí sí vas a usar el mismo chat para todos los pasos, a propósito: cada respuesta sirve de contexto para la siguiente.

1. Pedido de una sola vez. En un chat nuevo envía:

Crea un sistema de inventario para una tienda.

2. Pedido por pasos. En otro chat nuevo envía el paso 1. Espera la respuesta, léela y recién envía el siguiente paso en el mismo chat:

Paso	Mensaje
1	Voy a crear un sistema de inventario para una tienda pequeña en Java. Lista los 5 requisitos principales del sistema.
2	Con esos requisitos, diseña las clases necesarias. Para cada clase indica sus atributos con su tipo de dato.
3	Escribe el código Java de la clase Producto con sus atributos, un constructor y los métodos get y set.
4	Revisa el código de la clase Producto y propone 3 mejoras concretas.

Resultado esperado: *El pedido de una sola vez produce algo genérico o muy largo. El pedido por pasos produce requisitos, un diseño de clases y un código de la clase Producto coherente con ese diseño.*

3. Compilar la clase (opcional). Si tienes el JDK, copia el código del paso 3 en un archivo Producto.java dentro de la carpeta del repositorio y ejecuta:

```
javac Producto.java
```

Resultado esperado: *El comando termina sin mensajes y aparece el archivo Producto.class. Si aparece un error, pégalo en el mismo chat y pide que lo corrija.*

4. Registrar en la bitácora una línea por paso indicando qué te entregó la IA y una línea comparando el resultado con el pedido de una sola vez.

Cuidado: *No confundas descomposición con Chain of Thought. En Chain of Thought la IA divide su razonamiento para resolver un problema; en la descomposición eres tú quien divide la tarea en varios pedidos.*

	Laboratorio de Diseño de Interfaces de Programación Avanzado	Página 14
---	---	-------------

Captura del resultado:

Diseño de Clases del Sistema (Paso 2)

Para estructurar el sistema de inventario en Java bajo el paradigma orientado al diseño de objetos, se definieron las siguientes 4 clases principales con sus atributos y tipos de datos:

1. Clase **Producto**

Representa cada artículo físico disponible en la tienda y gestiona sus datos individuales.

- **codigo** (**String**): Identificador único o código de barras.
- **nombre** (**String**): Nombre descriptivo del producto.
- **precioVenta** (**double**): Precio unitario al público.
- **stock** (**int**): Cantidad actual disponible en el almacén.
- **stockMinimo** (**int**): Límite para activar la alerta de reposición.

2. Clase **DetalleVenta**

Representa una línea específica dentro de una transacción de venta para asociar la cantidad de un producto.

- **producto** (**Producto**): El objeto producto que se está vendiendo.
- **cantidad** (**int**): Unidades compradas de ese artículo específico.
- **subtotal** (**double**): Valor calculado de la línea (cantidad multiplicada por el precio de venta).

3. Clase **Venta**

Registra la transacción comercial global realizada en la caja de la tienda.

- **idVenta** (**String**): Identificador único del ticket o factura.
- **fechaHora** (**LocalDateTime**): Fecha y hora exacta de la transacción.
- **detalles** (**List<DetalleVenta>**): Lista con los productos y cantidades compradas.
- **total** (**double**): Monto total acumulado a pagar por el cliente.

4. Clase **Inventario**

Centraliza la lógica de negocio, el control de stock, las alertas y la manipulación de los datos en memoria.

- **productos** (**Map<String, Producto>**): Estructura clave-valor (código-producto) para búsquedas instantáneas.
- **historialVentas** (**List<Venta>**): Registro acumulado de todas las ventas efectuadas para reportes.

Código Java de la Clase Producto (Paso 3)

A continuación se muestra la implementación inicial de la clase **Producto** con sus atributos, constructor y métodos get/set:

```
public class Producto {  
    private String codigo;  
    private String nombre;  
    private double precioVenta;  
    private int stock;  
    private int stockMinimo;  
  
    // Constructor  
    public Producto(String codigo, String nombre, double precioVenta, int  
stock, int stockMinimo) {  
        this.codigo = codigo;  
        this.nombre = nombre;  
        this.precioVenta = precioVenta;  
        this.stock = stock;  
        this.stockMinimo = stockMinimo;  
    }  
  
    // Métodos Getters y Setters  
    public String getCodigo() {  
        return codigo;  
    }  
  
    public void setCodigo(String codigo) {  
        this.codigo = codigo;  
    }  
  
    public String getNombre() {  
        return nombre;  
    }  
  
    public void setNombre(String nombre) {  
        this.nombre = nombre;  
    }  
}
```

```
public double getPrecioVenta() {  
    return precioVenta;  
}  
  
public void setPrecioVenta(double precioVenta) {  
    this.precioVenta = precioVenta;  
}  
  
public int getStock() {  
    return stock;  
}  
  
public void setStock(int stock) {  
    this.stock = stock;  
}  
  
public int getStockMinimo() {  
    return stockMinimo;  
}  
  
public void setStockMinimo(int stockMinimo) {  
    this.stockMinimo = stockMinimo;  
}  
}
```

Ejercicio 6: Prompt estructurado y autocrítica

Cuando el prompt es largo, conviene separar sus partes para que no se mezclen. Las etiquetas <rol>...</rol> no son código: son separadores y puedes elegir sus nombres. Después vas a pedirle a la IA que revise su propia respuesta.

1. Prompt básico. En un chat nuevo envía:

Dame casos de prueba para un login.

2. Prompt estructurado. En otro chat nuevo envía:

```
<rol>Actua como analista de pruebas de software.</rol>  
<contexto>Login web con correo y contraseña. La cuenta se bloquea  
despues de 3 intentos fallidos.</contexto>  
< tarea>Piensa paso a paso que puede fallar y escribe 6 casos de prueba.</ tarea>  
<formato>Tabla con las columnas: ID, escenario, datos de entrada,  
resultado esperado.</formato>
```

Resultado esperado: Una tabla con 6 casos de prueba y las 4 columnas pedidas, que incluye el bloqueo después de 3 intentos.

3. Pedir autocrítica. En el mismo chat del paso 2 envía:

Revisa tu tabla: faltan casos límite como campos vacíos, correo sin @ o contraseña con espacios? Agrega los que falten e indica cuales agregaste.

Aquí sí usas el mismo chat: quieres que la IA revise la tabla que acaba de escribir, así que debe estar en su ventana de contexto.

Resultado esperado: La IA agrega casos nuevos (por ejemplo, correo vacío o sin @) e indica cuáles son.

4. Evaluar. Revisa la tabla final y anota en tu bitácora:

Qué revisar	Cumple (Sí / No)
¿Tiene las 4 columnas pedidas?	SI
¿Incluye el bloqueo después de 3 intentos?	SI
¿Incluye casos con campos vacíos?	SI
¿Indica qué casos agregó en la autocrítica?	SI
¿Hay algún caso repetido o que no tenga sentido?	NO

Cuidado: La IA puede equivocarse incluso cuando se revisa a sí misma. Si ves un caso repetido o incorrecto, anótalo: tú eres el último revisor.

5. Guardar el prompt estructurado en tu bitácora, dentro de un bloque de código:

```
```text
(pega aquí tu prompt estructurado y el mensaje de autocritica)
```
```

Captura del resultado:

| Qué revisar | Cumple (Sí / No) |
|--|------------------|
| ¿Tiene las 4 columnas pedidas? | SI |
| ¿Incluye el bloqueo después de 3 intentos? | SI |
| ¿Incluye casos con campos vacíos? | SI |
| ¿Indica qué casos agregó en la autocrítica? | SI |
| ¿Hay algún caso repetido o que no tenga sentido? | NO |

Ejercicio 7: Publicar la bitácora en GitHub

Tu bitácora solo existe en tu computadora hasta que haces push. Publicarla la convierte en la evidencia de este laboratorio.

1. **Enlazar la bitácora desde el README.** Agrega esta línea al README.md principal:

```
- [Bitacora de tecnicas avanzadas](prompts/BITACORA.md)
```

2. **Ver qué cambió.**

```
git status
```

Resultado esperado: *modified: README.md y la carpeta prompts/ como carpeta sin seguimiento (y Producto.java si hiciste el paso opcional del Ejercicio 5).*

3. **Registrar y subir el cambio.**

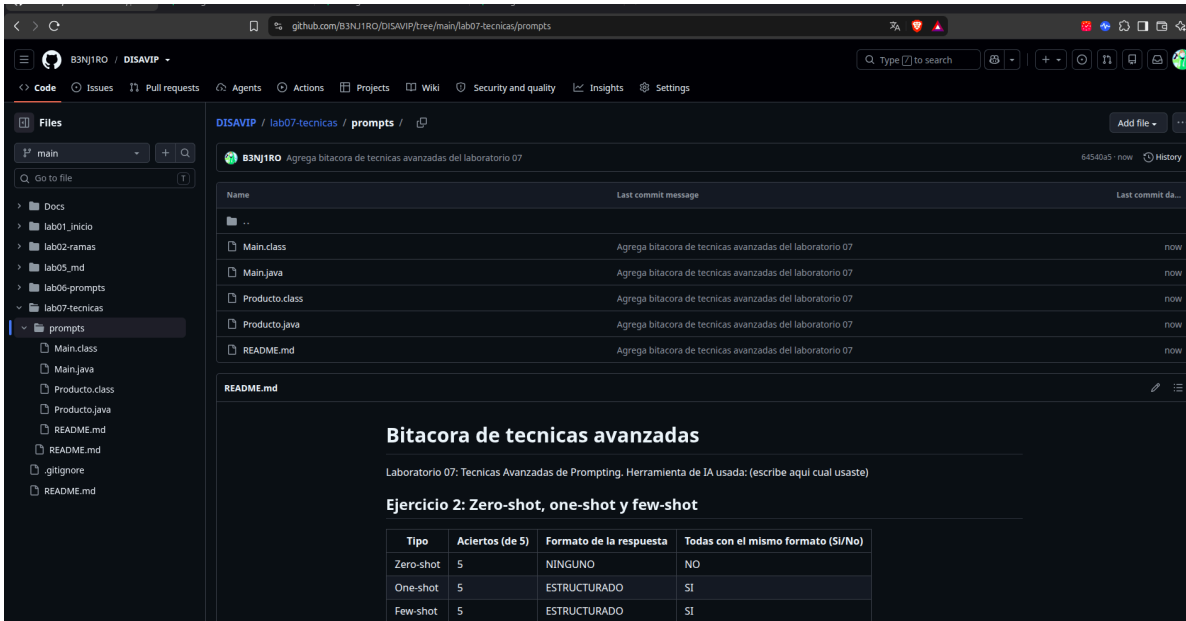
```
git add .
git commit -m "Agrega bitacora de tecnicas avanzadas del laboratorio 07"
git push origin main
```

Resultado esperado: *Varias líneas de progreso y, al final, una que termina en main -> main.*

4. **Recargar tu repositorio en GitHub** y hacer clic en el enlace Bitacora de tecnicas avanzadas.

Resultado esperado: *La bitácora se abre renderizada: títulos, tablas completas y los prompts dentro de bloques de código.*

Captura de la bitácora publicada:



The screenshot shows a GitHub repository for 'B3NJ1RO / DISA VIP'. The commit history shows a series of commits adding files to the 'lab07-tecnicas' directory. The README file, titled 'Bitacora de tecnicas avanzadas', contains the following content:

Laboratorio 07: Tecnicas Avanzadas de Prompting. Herramienta de IA usada: (escribe aquí cual usaste)

Ejercicio 2: Zero-shot, one-shot y few-shot

| Tipo | Aciertos (de 5) | Formato de la respuesta | Todas con el mismo formato (SI/No) |
|-----------|-----------------|-------------------------|------------------------------------|
| Zero-shot | 5 | NINGUNO | NO |
| One-shot | 5 | ESTRUCTURADO | SI |
| Few-shot | 5 | ESTRUCTURADO | SI |

LINK: [LAB07 - PROMPTS](#)

Si algo se te complica: el botón de pánico

```
git checkout prompts/BITACORA.md
```

Este comando devuelve la bitácora a como estaba en el último commit. Y si la IA se "enreda" en una conversación larga, abre un chat nuevo y envía el prompt completo desde el inicio.

TAREA:

DESARROLLO INDIVIDUAL. Resuelve por tu cuenta, en el mismo repositorio lab07-tecnicas que creaste en la parte guiada. Reserva los últimos minutos para las observaciones, las conclusiones y la entrega.

Diseñar un prompt avanzado que combine al menos tres técnicas

Elige una tarea de software que te resulte útil (por ejemplo: generar casos de prueba para un registro de usuarios, explicar un error de compilación, diseñar las clases de un sistema de notas o documentar una función) y crea el archivo prompts/TAREA.md. En él vas a documentar cómo llevaste un prompt básico hasta un prompt avanzado, iterando al menos tres veces.

La solución debe cumplir:

- Un archivo prompts/TAREA.md publicado en GitHub y renderizado correctamente.
- Tres versiones del prompt (v1, v2 y v3), cada una dentro de un bloque de código text.
- Para cada versión: qué técnica agregaste, por qué y qué mejoró en la respuesta.
- Un prompt final que combine al menos tres técnicas de la sesión: few-shot, chain of thought, role prompting, descomposición, prompt estructurado o autocrítica.

- Un rol específico (no "experto") y un formato de respuesta definido.
- Una tabla que indique qué parte del prompt final corresponde a cada técnica.
- Una tabla de evaluación del resultado con al menos cuatro criterios (Sí / No).
- Un párrafo que explique por qué elegiste esas técnicas para tu tarea y no otras.
- Un enlace desde el README.md principal hacia prompts/TAREA.md.
- Al menos dos commits con mensajes descriptivos, ambos publicados con push.

1. Crear el archivo con esta estructura:

```
# Tarea: Mi prompt avanzado

## Tarea elegida

## Version 1: prompt basico

## Version 2

## Version 3: prompt final

## Tecnicas usadas en el prompt final

## Evaluacion del resultado

## Por que elegi estas tecnicas
```

2. Iterar con la vista previa abierta.

No escribas el prompt final directamente. Envía la versión 1, evalúa lo que falta, agrega una técnica y vuelve a enviar. La tarea no evalúa solo tu prompt final: evalúa que se note cómo llegaste a él y por qué elegiste cada técnica.

3. Enlazar la tarea desde el README principal.

```
- [Tarea: mi prompt avanzado](prompts/TAREA.md)
```

4. Publicar todo.

```
git add .
git commit -m "Agrega tarea de prompt avanzado"
git push origin main
```

5. Verificar en GitHub. Recarga tu repositorio y comprueba las cinco cosas:

| Qué revisar | Dónde se comprueba |
|--|---------------------------------------|
| El README tiene los enlaces a la bitácora y a la tarea | Portada del repositorio |
| La bitácora tiene todas sus tablas completas | Archivo prompts/BITACORA.md en GitHub |
| La tarea muestra las tres versiones del prompt | Archivo prompts/TAREA.md en GitHub |
| La tabla de técnicas identifica al menos tres técnicas | Archivo prompts/TAREA.md en GitHub |

| Qué revisar | Dónde se comprueba |
|--|---------------------------------|
| Los commits tienen mensajes que se entienden | Pestaña Commits del repositorio |

Colocar capturas de las iteraciones y del resultado:

[Pega aquí tu captura]

La respuesta de la IA a tu prompt final

[Pega aquí tu captura]

El archivo prompts/TAREA.md renderizado en GitHub

Entregable: enlace a tu repositorio en GitHub, con prompts/BITACORA.md y prompts/TAREA.md publicados y renderizados.

Responde las siguientes preguntas:

a) En el Ejercicio 2, ¿qué diferencia observaste entre las respuestas de zero-shot y few-shot? ¿Por qué con los ejemplos el formato se mantuvo igual?

b) En el Ejercicio 3, ¿por qué es útil que la IA muestre sus pasos aunque la respuesta directa haya sido correcta?

c) ¿En qué se diferencian la descomposición y Chain of Thought? Usa lo que hiciste en los Ejercicios 3 y 5.

d) En el Ejercicio 6, ¿la autocrítica encontró casos que faltaban? ¿Por qué, aun así, tú debes revisar la respuesta final?

Errores frecuentes y cómo resolverlos

| Mensaje o síntoma | Solución |
|--|--|
| En zero-shot cada respuesta tiene un formato distinto | Es normal. Usa few-shot: agrega de 2 a 5 ejemplos con el formato exacto que quieres. |
| En few-shot la IA no respeta el formato | Revisa que todos tus ejemplos tengan el mismo formato y agrega "Responde solo con el formato de los ejemplos". |
| La IA copia los ejemplos en vez de clasificar los comentarios nuevos | Separa claramente los ejemplos de los comentarios a clasificar, con una línea en blanco o un título. |
| El resultado del Ejercicio 3 no coincide con S/ 318,60 | Revisa el orden: primero el descuento y después el IGV sobre el precio con descuento. Si la IA se equivocó, anótalo: es justamente lo que Chain of Thought permite detectar. |
| La IA responde en inglés | Agrega al final del prompt: "Responde en español". |
| Con rol y sin rol la respuesta es casi igual | El rol es demasiado vago. Indica qué especialista es y para quién explica. |
| En la descomposición la IA olvida lo que dijo antes | Envíaste los pasos en chats distintos. Todos los pasos van en el mismo chat. |
| error: class Producto is public, should be declared in a file named... | El archivo no se llama exactamente Producto.java (revisa mayúsculas). |
| javac no se reconoce como comando | Falta instalar el JDK (gratuito, adoptium.net) o no reiniciaste la terminal después de instalarlo. Este paso es opcional. |
| La IA muestra las etiquetas <rol> en su respuesta o las comenta | No es un error: las etiquetas solo separan partes. Si te molesta, usa títulos ## Rol, ## Tarea en su lugar. |
| La autocritica dice que todo está bien, pero ves errores | La IA también se equivoca al revisarse. Señálale el error concreto en el mismo chat y pide la corrección. |
| En una conversación larga las respuestas empeoran | La ventana de contexto está saturada. Abre un chat nuevo con un resumen de lo importante. |
| La IA dice que alcanzaste el límite de mensajes | Es el límite del plan gratuito. Espera unos minutos o continúa en otro asistente gratuito y anótalo en las observaciones. No pagues ningún plan. |
| git push pide usuario y contraseña | GitHub no acepta tu contraseña normal. Inicia sesión desde la ventana del navegador que abre Git o usa un token personal. |

OBSERVACIONES:

(Las observaciones son las notas aclaratorias, objeciones y problemas que se pudo presentar en el desarrollo del laboratorio)

CONCLUSIONES:

(Las conclusiones son una opinión sobre tu trabajo, explicar cómo resolviste las dudas o problemas presentados en el laboratorio. Además una opinión crítica de lo realizado)
